

AVIRAT.AI

Internship Training Programme

Agentic AI Engineering Track

Intern	Deeya Mehta
Role	AI Engineer Intern
Year	3rd Year, B.Tech Computer Science
Specialisation	Agentic AI — AI Agents, RAG, Multi-agent Systems
Duration	45 Days (Weeks 1 - 6)
Prepared by	Gaurav Sharma, Data Engineer — Avirat.ai
Date	June 2026

1. Programme Overview

Deeya is a 3rd-year Computer Science student with a specific interest in Agentic AI. This 45-day programme is built from the ground up around that goal: every week is a deliberate step toward the capstone — a fully orchestrated multi-agent system she can deploy, demo, and discuss confidently in any technical interview.

Foundations are covered in the first two weeks and moved through quickly. The majority of time — Weeks 3 through 6 — is spent in the LLM, RAG, and agentic layers. Week 4 deliberately builds an agent by hand, without frameworks, so that when frameworks are introduced in the capstone phase she understands exactly what they are abstracting.

1.1 Programme Structure

Phase	Days	Theme	Key Deliverable
2.1 Foundations	1-14	Python · APIs · Git · SQL	AI News Digest FastAPI service
2.2 LLMs & RAG	15-21	Embeddings · ChromaDB · Retrieval	Document Q&A RAG pipeline
2.3 Agent Fundamentals	22-28	ReAct · Tool Use · Memory	Hand-built Research Agent
2.4 Agentic Systems	29-45	Multi-agent · HITL · Docker	Multi-agent capstone app

1.2 Technology Stack

Phase	Core Libraries	AI / Agent Stack	Storage & Infra
2.1	Python 3.11, asyncio, httpx, FastAPI, Pydantic	OpenAI / Anthropic Chat Completions API	SQLite, Git, GitHub
2.2	sentence-transformers, PyPDF2, tiktoken	OpenAI Embeddings or HuggingFace models	ChromaDB, FastAPI SSE
2.3	Python stdlib (no framework)	OpenAI / Anthropic tool use, DuckDuckGo API	SQLite memory, JSONL trace logs
2.4	LangGraph or CrewAI, Docker, HTML/JS	Multi-agent orchestration, HITL patterns	ChromaDB, SQLite, Docker Compose

1.3 Evaluation Schedule

Phase	Primary Check	Technical Check	Review Format
2.1 — Days 1-14	FastAPI service + async patterns	Git workflow & SQL schema	Live endpoint demo
2.2 — Days 15-21	RAG pipeline hit rate $\geq 4/5$	Vector storage & retrieval	Notebook walkthrough
2.3 — Days 22-28	Agent completes task end-to-end	Tool calling & memory	Live agent run
2.4 — Days 29-45	Multi-agent capstone demo	HITL & orchestration logic	Presentation + Q&A

2. Training Modules

Each module below covers one phase of the programme. Each phase contains a learning task list, a numbered mini-project, a deliverables list, and evaluation criteria. All projects are cumulative — later projects build on earlier ones.

2.1 Python Mastery & AI Engineering Foundations Days 1-14

Deeya arrives with two years of CS coursework. This phase moves quickly through production-grade Python, async programming, API consumption, and the developer tooling that underpins every agentic system she will build. By Day 14 she can write, test, and ship a clean Python service independently.

Focus areas: *Async Python · REST APIs · Git Discipline · SQL for Agent State*

2.1.1 Learning Tasks

- Advanced Python: OOP, decorators, context managers, list/dict comprehensions
- Async Python: asyncio, async/await, aiohttp — essential for concurrent agent calls
- Git & GitHub: feature branches, pull requests, rebasing, descriptive commit messages
- Linux CLI: navigation, permissions, virtual environments, environment variables
- REST API consumption: requests/httpx, auth headers, pagination, rate limits
- FastAPI: define endpoints, Pydantic request/response models, dependency injection
- SQL: CRUD, JOINS, GROUP BY, subqueries — agents need persistent state storage
- Code quality: PEP8, flake8, type hints, modular function design
- Project structure: separating config, services, models, and route files
- JSON & secrets management: .env files, python-dotenv, never committing keys

Mini Project 1 — CLI AI News Digest Tool

1. Fetch headlines from a public news API (NewsAPI or HackerNews)
2. Store articles in SQLite: schema with id, title, url, summary, fetched_at
3. Call OpenAI / Anthropic API to summarise each article in two sentences
4. Expose a FastAPI endpoint GET /digest returning today's summaries as JSON
5. All external API calls must use async/await — no blocking I/O anywhere
6. Bonus: add GET /search?q= endpoint that queries the SQLite store by keyword

2.1.2 Deliverables

- GitHub repository with feature branch workflow and pull requests
- FastAPI app with /digest and /search endpoints — both returning correct JSON
- SQLite schema file + seed/migration script
- requirements.txt, .gitignore, and .env.example committed
- README with setup steps and sample curl commands

2.1.3 Evaluation Criteria

- Both endpoints return correct, well-structured JSON responses
- Async pattern used correctly — no synchronous blocking calls present
- Git log shows clean branch history and descriptive commit messages
- SQL schema is normalised; queries execute without errors
- Codebase passes flake8 lint with zero warnings

2.2 LLMs, Embeddings & RAG Pipelines Days 15–21

Before building agents, Deeya must understand what agents are built on top of. This week covers LLM API patterns in depth, embedding models, vector databases, and retrieval-augmented generation — the memory and knowledge layer of any serious agentic system.

Focus areas: *LLM APIs · Semantic Search · Vector Storage · Prompt Engineering*

2.2.1 Learning Tasks

- LLM API deep dive: chat completions, system prompts, token limits, streaming
- Prompt engineering: zero-shot, few-shot, chain-of-thought, role prompting
- Tokenisation: how text becomes tokens; cost and context window implications
- Embeddings: what they represent; generating via OpenAI or sentence-transformers
- Vector databases: ChromaDB locally; concepts behind Pinecone / Weaviate
- RAG pattern end-to-end: chunk → embed → store → retrieve → augment → generate
- Chunking strategies: fixed-size, sentence-boundary, semantic chunking
- Retrieval evaluation: precision@k, recall, MRR — measuring if RAG actually works
- Streaming LLM responses: server-sent events (SSE) in FastAPI
- Cost awareness: token counting, batching embeddings, caching repeated responses

Mini Project 2 — Document Q&A RAG Pipeline

7. Accept any PDF or plain-text file as input to the pipeline
8. Chunk the document and embed each chunk using sentence-transformers or OpenAI
9. Store vectors in ChromaDB with metadata: source filename, page number, chunk ID
10. On query: embed the question → retrieve top-k chunks → feed to LLM with context
11. Expose as FastAPI endpoint: POST /ask {doc_id, question} → {answer, sources}
12. Evaluate: prepare 5 questions with known correct answers; report hit rate
13. Bonus: compare two chunking strategies; report which retrieves more accurately

2.2.2 Deliverables

- RAG pipeline FastAPI service committed to GitHub
- ChromaDB integration working with metadata filtering
- Evaluation report: 5-question hit-rate test results
- README with architecture description and chunking strategy rationale

2.2.3 Evaluation Criteria

- Pipeline correctly answers at least 4 of 5 evaluation questions
- Vector storage and retrieval code is modular and reusable
- SSE streaming endpoint delivers tokens progressively to client
- Chunking strategy choice is justified with evidence in README

2.3 Agent Fundamentals — ReAct, Tool Use & Memory Days 22-28

This is the core week. Deeya builds her first real agent from scratch — a reasoning loop that decides which tool to call, observes the result, and continues until the task is complete. No framework shortcuts yet. Understanding the mechanics at this level makes every future framework far easier to master.

Focus areas: *ReAct Loop · Tool Calling API · Memory Architecture · Agent State*

2.3.1 Learning Tasks

- Agent architecture: the Perceive → Think → Act loop in detail
- ReAct pattern: Reasoning + Acting — thought / action / observation cycles
- OpenAI function calling: defining tool schemas as JSON, parallel tool calls
- Anthropic tool use API: tool_use content blocks, tool_result response pattern
- Wrapping any Python function as a typed, callable agent tool
- Memory taxonomy: in-context (short-term), external DB (long-term), episodic
- Implementing conversation memory: summarisation buffer, sliding window, vector retrieval
- Error handling in agent loops: tool failures, infinite loop guards, max-iteration budget
- Structured output: forcing agents to return valid JSON matching a Pydantic schema
- Agent observability: logging every thought / action / observation step to a trace file

Mini Project 3 — Research Assistant Agent (No Framework)

14. Four tools the agent must support: web_search, calculator, read_file, write_file
15. Implement the full ReAct loop: reason → call tool → observe → reason again
16. Conversation memory: context must persist correctly across 10+ turn sessions
17. Guard rails: maximum 10 iterations; graceful error handling on tool failures
18. Structured final answer: agent must return {answer, sources, confidence} JSON
19. Trace logger: save every thought / action / observation turn to a .jsonl file
20. Bonus: add a sqlite_query tool so the agent can interrogate a database

2.3.2 Deliverables

- Research agent Python package on GitHub
- Four working tools: web_search, calculator, read_file, write_file
- Conversation memory that persists across sessions
- Trace .jsonl log file from a complete test run, committed to the repo
- README with agent loop architecture description

2.3.3 Evaluation Criteria

- Agent completes a realistic multi-step research task end-to-end
- Tool-calling handles errors without crashing the main loop
- Memory correctly maintains context across 10+ conversation turns
- Trace log shows the complete thought → action → observation chain
- Max-iteration guard triggers and exits gracefully when reached

2.4 Agentic Systems — Multi-Agent, Orchestration & Capstone Days 29–45

The capstone phase. Deeya moves from a single agent to a coordinated multi-agent system using LangGraph or CrewAI. She builds a production-ready agentic application, adds evaluation and human-in-the-loop controls, containerises it, and ships a documented, demo-ready system.

Focus areas: *Multi-Agent Orchestration · LangGraph / CrewAI · HITL · Docker Deployment*

2.4.1 Learning Tasks

- Multi-agent design patterns: Orchestrator-Worker, Supervisor, Peer-to-Peer networks
- LangGraph: StateGraph, typed nodes, edges, conditional routing, checkpointing
- CrewAI: Agents, Tasks, Crew, sequential and hierarchical process types
- Agent specialisation: planner, executor, researcher, critic/reviewer roles
- Human-in-the-loop (HITL): approval checkpoints, interrupt/resume patterns
- Agent evaluation: task completion rate, tool call accuracy, answer quality scoring
- Streaming multi-agent outputs to a frontend via SSE or WebSockets
- Docker basics: write a Dockerfile, build an image, run via docker-compose
- Guardrails: output validation, content filtering, per-agent loop budgets
- Portfolio preparation: explain system architecture in a 5-minute structured talk
- GitHub profile polish: profile README, pinned repositories, contribution graph

Capstone — AI Research & Report Generation System

21. Planner Agent: decomposes the user's query into ordered research sub-tasks
22. Researcher Agent: uses the Week 3 RAG pipeline plus a live web search tool
23. Writer Agent: synthesises all findings into a structured markdown report
24. Critic Agent: reviews the draft and requests targeted revisions if quality is low
25. HITL checkpoint: user must explicitly approve before the report is saved
26. FastAPI backend + minimal HTML/JS frontend: submit query → stream agent progress → download report
27. Deployable: entire system starts with a single docker-compose up command
28. Evaluation: run 5 test queries; score each report on accuracy, completeness, and format
29. 5-minute recorded demo video linked from the README

30. Full README: architecture diagram, agent role descriptions, setup guide, known limitations

2.4.2 Deliverables

- Public GitHub repository with complete README and architecture diagram
- Multi-agent system (LangGraph or CrewAI) running end-to-end
- FastAPI backend and simple HTML/JS frontend
- docker-compose.yml — system starts with one command
- Evaluation report: 5 test queries with scores and observations
- Recorded demo video (5 minutes)
- Updated GitHub profile with pinned repositories and profile README

2.4.3 Evaluation Criteria

- Live demo: system completes a research task from query to final report
 - All four agent roles are present and observable in the trace logs
 - HITL checkpoint pauses execution and correctly waits for user approval
 - docker-compose up starts the full system cleanly without manual steps
 - Evaluation report includes honest scores and notes on limitations
 - Architecture diagram correctly describes agent roles and data flow
 - Intern can confidently answer Q&A on design choices and tradeoffs
-

3. Day 45 Final Evaluation Checklist

Use this checklist during the final review session. Each item must be verified live — either by running the code or walking through the repository together. Items that cannot be demonstrated are marked as incomplete.

☐	All mini-project repositories present, cloned, and running without errors
☐	Git workflow demonstrated: branches, descriptive commits, and pull requests throughout
☐	Week 1-2: /digest and /search FastAPI endpoints return correct async JSON
☐	Week 1-2: SQLite schema normalised; no blocking I/O in async routes
☐	Week 3: RAG pipeline correctly answers at least 4 of 5 evaluation questions
☐	Week 3: ChromaDB integration working; chunking strategy justified in README
☐	Week 4: Research agent completes a multi-step task end-to-end without crashing
☐	Week 4: Trace .jsonl log shows complete thought → action → observation chain
☐	Week 4: Conversation memory persists correctly across 10+ turns
☐	Week 4: Max-iteration guard triggers and exits gracefully
☐	Capstone: all four agent roles present and observable in logs
☐	Capstone: HITL checkpoint implemented, tested, and demonstrated
☐	Capstone: docker-compose up starts the system cleanly
☐	Capstone: 5-query evaluation report with scores committed to repo
☐	Capstone: recorded demo video (5 min) linked from README
☐	GitHub profile polished: profile README, pinned repos, clean commit history
☐	requirements.txt and .gitignore present in every project repository
☐	Intern can explain agent loop design, tool calling, and orchestration decisions